Experiment No: 08

Student Name: Amol Saurav
UID: 23BCS10067
Branch: BE-CSE
Section/Group: KRG_2B
Semester: 6th
Date of Performance: 08/04/2026
Subject Code: 23CSH-314
Subject Name: System Design

Aim:

To design and analyze a scalable stock trading application that enables users to buy and sell stocks in real time, view stock price and history in real time ensuring high availability, low latency, and reliability at large scale.

Objective:

- To understand system design principles for large-scale stock trading systems.
- To design APIs for users, market_data, funds, trading, portfolio and watchlist.
- To identify functional and non-functional requirements.
- To apply distributed system concepts such as CAP theorem.
- To design a high-availability and low-latency system architecture.

Procedure:

- Identify functional requirements of a ride-sharing application.
- Define non-functional requirements such as scalability, latency, and availability.
- Analyze CAP theorem trade-offs for real-time ride matching.
- Identify core entities such as users, stocks, orders, trade and portfolio.
- Design the system architecture using Draw.io.
- Validate the design for real-time stock trading scenarios.

Functional Requirements:

- Users should be able to do the registration, login and KYC verification.
- Application should allow users to view real-time stocks prices & its historical data.
- Application should support placing, modifying, and cancelling orders along with limiting orders with accurate status updates.
- Application should provide users with a watchlist with real-time updates.
- Application should provide a dashboard to track current holdings, trade history, P & L, and performance insights.

Core Entities of the System:

- Users
- Stocks
- Orders
- Trade
- Portfolio
- Watchlist

API Design:

Users

- POST: /auth/signup
- POST: /auth/login
- POST: /auth/verify-KYC

Market Data

- WS: /api/v1/stocks — Listing of all stocks
- GET: /api/v1/stocks/{stock_symbol_id} — Stock details
- GET: /api/v1/stocks/{stock_symbol_id}/history — Historical data

Funds

- POST: /api/v1/funds/deposit
- POST: /api/v1/funds/withdraw
- GET: /api/v1/funds/history

Trading

- POST: /api/v1/orders — Place buy/sell order
- GET: /api/v1/{user_id}/orders — List user orders
- GET: /api/v1/orders/{order_id} — Order status
- DELETE: /api/v1/orders/{order_id} — Cancel order

Portfolio

- GET: /api/v1/portfolio
- GET: /api/v1/portfolio/holdings
- GET: /api/v1/portfolio/positions
- GET: /api/v1/portfolio/performance — User ID sent in header for security

Watchlist

- GET: /api/v1/watchlists
- POST: /api/v1/watchlists/{watchlist_id}/symbols

Non-Functional Requirements:

Scalability

- Must support 2–3 million daily active users (DAU).
- Handle high-frequency trades across 8–10k stocks.
- Kafka broker and WebSocket gateway must scale horizontally to manage millions of stock price events.

Consistency vs Availability (CAP Theorem)

- Consistency is prioritized over availability.
- Correctness of stock prices and trades is critical.
- Wrong balances, duplicate trades, or stale prices are unacceptable.
- Real-time stock price delivery must remain highly available.

Latency

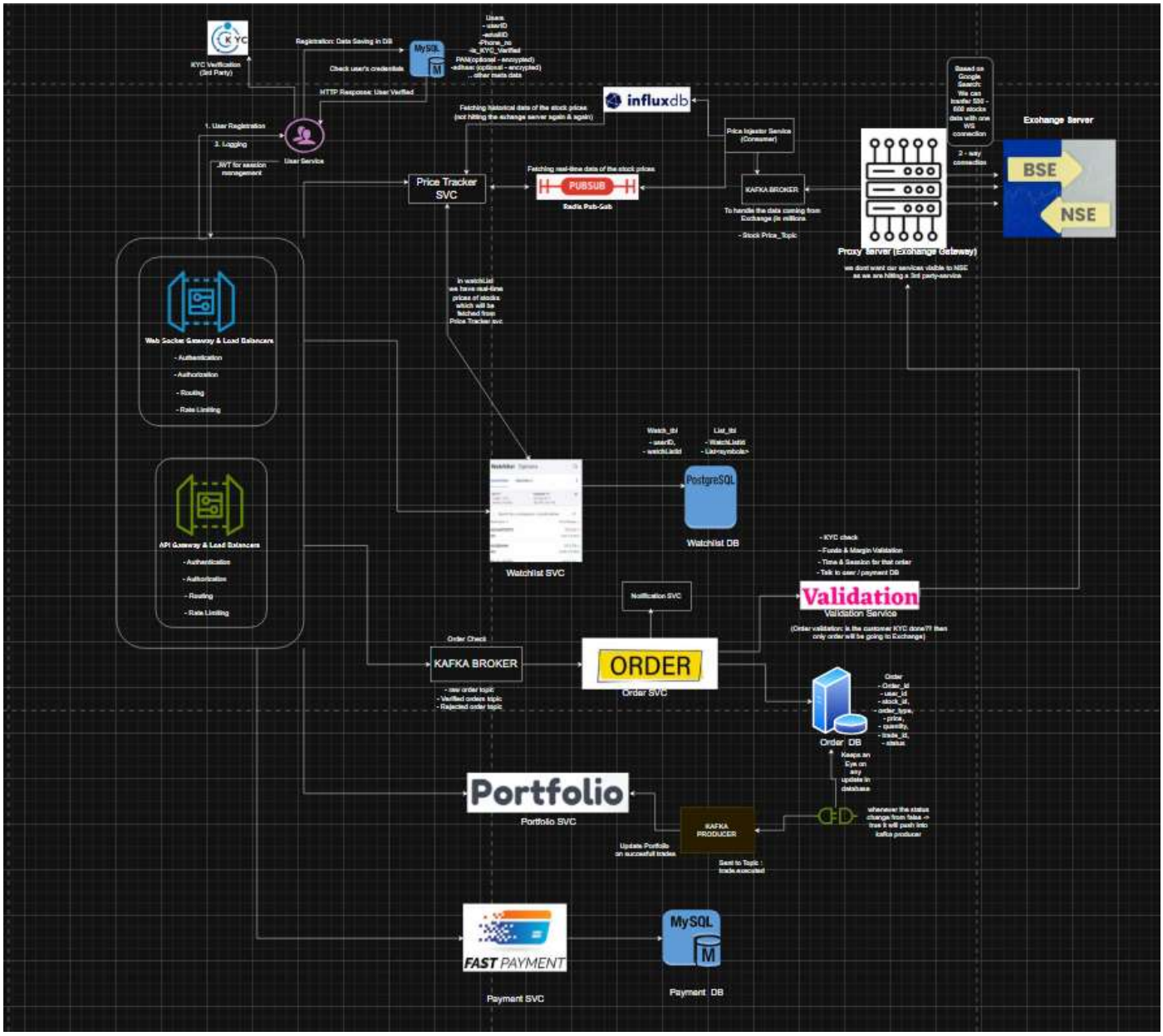
- Order placement should complete in under 100ms.
- Market data updates must be delivered in under 50ms.
- WebSocket connections must ensure low-latency bid/ask updates.

Reliability & Fault Tolerance

- API Gateway and Load Balancers must enforce authentication, authorization, routing, and rate limiting.
- Failover mechanisms required for Order Service, Payment Service, and Price Tracker Service.
- Redis Pub-Sub and Kafka topics must guarantee message durability and replay.

High Level Design: -

The system consists of Client Applications, an API Gateway, and core microservices such as User Service, Order Service, Portfolio Service, Watchlist Service, Payment Service, and Price Tracker Service. Load Balancers handle authentication, authorization, routing, and rate limiting. A Validation Service ensures compliance by verifying KYC, funds, and margins before orders are sent to the exchange. Real-time stock data flows through the Exchange Gateway into Kafka, enabling asynchronous processing and low-latency updates via Redis Pub-Sub and WebSocket Gateway. Each service uses dedicated databases, ensuring scalability, reliability, and secure data management.



Outcome:

The stock trading system was successfully designed with key features like buying and selling stocks, viewing the history of stocks, generating portfolio, ensuring scalability, low latency, and high availability for real-world usage.